

OSS-Next Completion Audit

Requirements & Task Tracker — regenerated 2026-07-18

5. Latest Audit Results — 2026-07-18

Dimension	Score	2026-07-17	2026-07-15
Overall (blended)	~79%	~75%	~65%
Functional Completeness (8 areas)	~87%	~81%	~62%
E-Commerce Feature Coverage (41 items)	~62%	~62%	~65%
Core Web Vitals / PageSpeed / SEO	~89%	~82%	~68%

Functional Completeness and CWV/SEO both moved up again — reflects the homepage A/B/C rebuild (real `middleware.ts` + `VariantHero` Suspense, replacing the crawlable `/version2` / `/version3` duplicate-content routes), the wishlist feature, PDP social sharing, the checkout/cart metadata fixes, the checkout consent-link and thumbnail fixes, and the `braintree` / `cheerio` named-import fixes — all shipped between the two audits. E-Commerce Feature Coverage holds flat at ~62% despite two genuine feature flips (**wishlist** and **social sharing** both went from  to ) — those +2 points were offset by scoring several previously-generous items more conservatively this pass: the address book turns out to be exactly one billing + one shipping address baked into the profile, not a true multi-address book; and order history has no shipment/carrier tracking number, only order-status display. Nothing that worked before stopped working — this is stricter grading, not regression, same pattern as the 07-15→07-17 dip. Full per-area/per-item breakdown from this pass lives in the 3 parallel research reports that produced it (not persisted verbatim here, to avoid this file going stale) — §6 below is the actionable distillation, with every genuinely NEW finding from this pass folded in alongside the existing punch list.

6. Current Task List

Living punch list, regenerated after each audit. Check items off as they're completed; don't delete completed items until the next full regeneration (so progress is visible).

High priority

- ☒ **Fix the account-update endpoint — it was a wrong URL/verb on our side, not a missing backend feature.** Found 2026-07-15 while investigating whether `edit-address` could be built for real: `services/user.service.ts` 's `ACCOUNT_DETAILS_URL ( api/auth/account-details/ )` 404s against the real backend in every HTTP method tried. Pulled the complete registered `api/auth/` route list from a Django debug 404 page, which confirmed no `account-details` route exists — initially logged as a backend gap. **Correction:** the user supplied `MY_ACCOUNT_PROFILE.md`, extracted from the original WordPress app's real working profile-update flow, which revealed the actual contract: `PUT` (not `PATCH`) to the `*same*` `api/auth/profile` URL the working GET already uses (no trailing slash), as a **full-object replace** (every profile field resent every time, not a diff). Verified live against the real backend: 200, data genuinely persists. *Done 2026-07-15 — `updateAccountDetails()` in `services/user.service.ts` now PUTs `api/auth/profile` with the full name+email+profile payload; `AccountDetailsForm.tsx` resends the user's current full `profile` unchanged alongside its own edits, and the new `edit-address` page (see below) does the mirror image, so neither form can wipe out what the other saved. Dropped the "Display name" field from `AccountDetailsForm` entirely — the real backend has no such field, it was always a dead input. Live- verified end-to-end (name-only save → address-only save → re-fetch → restore), confirming each save preserves the fields the other form owns and data genuinely persists. Typecheck + lint clean.*
- ☐ **Fix the Django backend checkout crash.** `/api/orders/checkout` throws `ValueError: Currency formatting is not possible using the 'C' locale` in `app/orders/views.py` line 89 (`locale.currency(...)` with no server locale configured) — on EVERY call, and the order saves before the crash. Blocks real checkout entirely once Braintree credentials exist. **Backend fix, not fixable here.**
- ☒ **Add `sitemap.ts` and `robots.ts`** covering all static routes plus dynamic ES-backed PDP/PLP URLs. Currently zero SEO route declarations exist. *Done 2026-07-15 — new `getAllProductHandles()` in `search.service.ts` paginates via `search_after` (ES's 10,000-result window is smaller than the real catalog). Verified live: `/sitemap.xml` returns exactly 10,267 URLs (3 static + 10,264 real published products, matching a direct ES count), with real `Lastmod` dates. `/robots.txt` disallows `/my-account`, `/cart`, `/checkout`. Typecheck + lint clean.*

✓ **Add `error.tsx`, `not-found.tsx`, and `global-error.tsx`** for the in-scope areas — none exist anywhere in the app today. Done 2026-07-15 — all 3 added at the true `app/` root (the only other route group, `(admin)`, is entirely unbuilt placeholder folders, so no separate scoped versions needed yet). Styled to match the site's existing empty-state pattern (icon + heading + description + CTA), with a phone-support link consistent with the rest of the site. Verified live: a nonexistent `/product/{handle}` correctly renders `not-found.tsx`'s content. **Known caveat, pre-existing, not introduced by this change:** the PDP's `notFound()` call happens inside a `<Suspense>` boundary, so the HTTP response has already started streaming with a 200 status by the time it fires — the page shows the correct not-found content, but the status code stays 200 instead of 404 on that specific route. Fixing that would mean restructuring the PDP's `Suspense` placement, out of scope for just adding these files. `error.tsx` / `global-error.tsx` verified via typecheck/lint only (standard, low-risk Next.js file conventions; not easily triggerable without introducing a real fault).

✓ **Rebuild the `/cart` page's fake logic:** replace the `setTimeout`-based loading state with a real one, replace the hardcoded `"SAVE10"` client-side coupon check with a real backend-validated flow (or remove the coupon box if no coupon system is planned), and replace the flat `$195` delivery-fee constant with a real call to `/api/orders/get-total` (same endpoint checkout already uses) so the two pages agree. Done 2026-07-15 — removed the fake 1.1s `setTimeout` entirely (cart items are already available synchronously post-mount; no real async work was ever happening there). Removed the coupon/promo box outright rather than fake-validate it — confirmed via the e-commerce feature audit that zero coupon backend exists anywhere to validate against. Delivery fee/tax now come from a real, debounced call to `/api/orders/get-total` (same endpoint + `CartLineItem` payload shape as checkout); since the cart page collects no ZIP, shipping/tax legitimately come back 0 and the UI says "Calculated at checkout"/"At checkout" rather than implying they're free. "Refresh totals" now re-runs this real call instead of replaying a fake timer. Also fixed 2 bugs found while in this file: the empty-cart CTA linked to `/buy-shipping-containers`, which doesn't exist (real PLP route is `/sale-shipping-containers`); and the delivery-estimate banner said "2–4 business days," inconsistent with the "1–5 days" claim used everywhere else on the site. Verified live: real item renders in ~750ms (no artificial delay), promo box gone, both fixed links/copy confirmed, `get-total` resolves to the correct deferred state, manual refresh re-fetches for real, total amount matches exactly. Typecheck + lint clean (one legitimate `set-state-in-effect` suppression for the hydration-safe mount flag — same accepted pattern already used elsewhere in this codebase).

✓ **Fix the duplicate `<main>` landmark bug** — nearly every My Account page plus the PDP (`ProductDetail.tsx` / `AccessoryDetail.tsx`) render their own `<main>` nested inside the shared layout's `<main>`. Remove the nested ones. Done 2026-07-15 — swapped the nested `<main>` for `<div>` in all 11 affected files (9 my-account pages + `ProductDetail.tsx` + `AccessoryDetail.tsx`); verified exactly one `<main>` renders per page via Playwright, typecheck + lint clean.

✅ **Orphaned duplicate homepage drafts were live, crawlable, and hurt SEO.**

`app/(market)/(home)/version2/page.tsx` and `.../version3/page.tsx` rendered at real, reachable URLs `/version2` and `/version3` — near-identical content/metadata to the real homepage, hand-rolled JSON-LD, wrong `canonical: "/"`, and not excluded from `robots.ts` / `sitemap.ts`. **Correction:** these weren't abandoned scaffolding — the user confirmed they're needed for real homepage A/B/C testing, just built the wrong way (separate crawlable routes instead of one URL with server-side variant selection). *Done 2026-07-17* — replaced with the standard single-URL pattern: new `middleware.ts` assigns a sticky variant (1 / 2 / 3, 30-day cookie) per visitor on `/` and forwards it via an `x-ab-home-variant` request header (not read from the cookie directly, so the very first request — before the cookie exists — still gets a consistent variant). `(home)/page.tsx` reads that header in a new `VariantHero` component, wrapped in `<Suspense fallback={<HeroSkeleton />>}` since `headers()` is a dynamic API — this keeps the rest of the homepage statically 'use cache' -able, only the Hero slice becomes per-request. Deleted the `version2 / version3` route folders entirely — now one canonical URL, one JSON-LD block, one sitemap entry, no duplicate-content risk. Verified live: sticky cookie persists the same variant across repeat requests; 6 fresh (no-cookie) requests correctly randomized across all 3 variants; the assigned variant's real Hero content renders; `/version2 / /version3` now fall through to the existing WordPress-proxy catch-all like any other unknown path (confirmed this fallback behavior is pre-existing/unrelated, not a regression). Typecheck + lint clean on all changed files. **Not yet built:** real analytics/conversion tracking per variant — no experimentation or analytics library exists anywhere in this repo today, so this only covers correct variant delivery, not measurement. Wire in whatever analytics tool is chosen once one exists, rather than fabricating tracking calls with nothing to receive them.

✅ **Checkout page metadata was stale and factually wrong.** `app/(market)/checkout/page.tsx`'s metadata (title-tag description + OG copy) still read "Complete your shipping container **reservation...** ****No commitment required**" / "Reserve **your shipping container today**. No deposit, no commitment**." This described the old fake "reserve, not a purchase" flow — `CheckoutClient.tsx` itself was already updated to real "Place Order" / "your card is charged" copy (per `API_INTEGRATION_STATUS.md`), but the page-level metadata (what shows in search results and shared links) was never updated to match. *Done 2026-07-18* — rewrote both the meta description and OG copy to describe the real payment flow. Verified live.

✅ **`cart/page.tsx` had zero metadata export**, violating this project's own "every page.tsx must export metadata" rule. Low severity since `/cart` is already `robots.txt` -disallowed, but the explicit per-page `robots: { index: false }` (same defense-in-depth pattern checkout uses) was still missing. *Done 2026-07-18* — split the page the same way `/wishlist` was just built: a new server-shell `page.tsx` exporting real metadata (title, description, canonical, `robots: { index: false, follow: true }`), rendering the actual cart UI (moved as-is) from a new `_components/CartPageClient.tsx`. No behavior change. Verified live: `/cart` returns 200 with a real `<title>` tag now.

Medium priority

-
- ✓ **Decide the fate of the 3 inert My Account subpages** (`downloads` , `edit-address` , `payment-methods`) — build real functionality (address CRUD backed by the real profile API, etc.) or remove/hide the nav entries if not planned. *Done 2026-07-15* — `edit-address` rebuilt for real: a billing + shipping address form (+ phone), backed by the same fixed `PUT api/auth/profile` flow above, merging the user's current name/email so as not to wipe them. `downloads` and `payment-methods` have no real backend behind them (no digital products, no stored- payment-method API) — kept as unlinked stub route files but removed from `AccountSidebar.tsx` 's nav so users can't reach dead pages. Verified live: both pages render correctly, sidebar shows exactly Dashboard/Orders/Addresses/Account Details/Logout. Typecheck + lint clean.
-
- ✓ **Add JSON-LD `BreadcrumbList` to the PLP** — a visual breadcrumb exists but no structured data, despite this project's own SEO rule requiring it on listing pages. *Done 2026-07-17* — added alongside building the shared `<JsonLd>` component below, since a 4th one-off inline script would've just meant migrating it again later. Mirrors the visible breadcrumb exactly for both `pType` variants (Home > Product Pricing / Home > Container Accessories). Verified live: both variants render the correct `BreadcrumbList` with matching names/URLs.
-
- ✓ **Enrich the PDP's Product JSON-LD to match the real WordPress schema** — `sku` / `mpn` / `gtin` (gtin reuses sku, see §2), full `image` array, `itemCondition` (mapped from the real `condition` field), `additionalProperty` (Container Size, Grade, Height Type, Material, Payment Type, Delivery Location — container products only), real tare `weight` (from `lib/data/pdpShippingContainers.ts` , averaged when the source is a manufacturer-variance range), a rolling (not hardcoded) `priceValidUntil` , and real `shippingDetails` matching the site's stated 1-5 day delivery. `description` is now shared between `generateMetadata` and the JSON-LD so they can't drift apart. Price is confirmed `sale_price` (computed by `calculateProductPrice()` , same value shown everywhere else on the site). **Still missing** (real, not fabricated): `aggregateRating` (no `reviewCount` in the ES index) and `hasMerchantReturnPolicy` (see §2). Verified live against a generic product, a real-location product, and an accessory — container-only fields correctly appear/disappear. *Done 2026-07-15*.
-
- ✓ **Build a shared `<JsonLd>` component** (`components/shared/JsonLd.tsx`) and migrate the 3 existing hand-rolled inline JSON-LD blocks (home, PDP, checkout) to use it. *Done 2026-07-17* — thin wrapper around the `<script type="application/ld+json">` pattern all 4 spots (home, checkout, PDP, and the new PLP breadcrumb above) already used identically; migrated all 4 to it, no behavior change. Verified live: all 4 pages still emit byte-identical JSON-LD output through the shared component. Typecheck + lint clean (pre-existing unused-import warnings on the homepage are unrelated — commented-out sections awaiting the "decide the homepage's direction" item below).
-

-
- ✓ **Create `config/routes.ts`** with typed path constants; replace the ~30 hardcoded internal path string literals found across the app. Done 2026-07-17 — *scoped deliberately to routes this app itself owns (Home, PLP, PDP, Cart, Checkout, My Account + subpages); left WordPress-owned content pages (Locations, Privacy, Terms, the quote landing page, Navbar's mega-menu) as plain hardcoded strings, since this app doesn't own those slugs and encoding them here would misrepresent who's responsible for keeping them correct. Replaced ~35 call sites across layout components, all my-account pages/canonicals, cart/checkout, and dynamic `/product/{handle}` link sites. Found and fixed 2 real broken links along the way: the checkout empty-cart CTA linked to `/products` (doesn't exist) and the PDP's "View All Containers" linked to `/product` (doesn't exist) — both now correctly point at `/sale-shipping- containers` . Also fixed a latent `excludePaths` bug in `app/layout.tsx`/`ZipAutoDetect.tsx` (`'/account'` instead of `'/my-account'` — never matched, so the geolocation prompt was never actually excluded on My Account pages) and a typo (`pType=acesories` , missing an "s") in `ProductVariantShell.tsx` 's listing crumb — though live-tracing that one found it's in unreachable code: accessory products render via the separate `AccessoryDetail.tsx` , which has no breadcrumb at all (a real, separate gap — noted below, not fixed in this pass, since adding one is new feature work, not a routing-constants change). Verified live: all touched pages return 200, both fixed broken links confirmed pointing at the real PLP. Typecheck + lint clean (pre-existing, unrelated errors confirmed via diff: `AccountPageShell.tsx` / `Navbar.tsx` `set-state-in-effect`, `Footer.tsx`'s untouched `Locations` `<a>` tag).*
-
- ✓ Accessory PDPs (`AccessoryDetail.tsx`) have no breadcrumb back to the PLP at all, unlike container PDPs (`ProductVariantShell.tsx`). Found 2026-07-17 while tracing the `routes.ts` refactor above — not fixed there since it's a new UI addition, not a path-constant swap. Done 2026-07-17 — *added a Home / Container Accessories / {title} breadcrumb, matching `ProductVariantShell.tsx` 's exact visual style, linking to `ROUTES.PLP_ACCESSORIES` . Also fixed a real bug found in the same file: the "Call Us" button dialed a hardcoded `+18886780313` — a different number than the site's actual `CONTACT_NUMBER` (888-977-9085) used everywhere else — now uses the shared constant. Typecheck + lint clean; verified live (breadcrumb text + correct phone number both present in the real rendered page).*
-
- ✓ **Wire real Suspense fallbacks on the PLP.** Both boundaries in `sale-shipping-containers/page.tsx` currently have no `fallback` at all — and a fully-built, dimension-matched `PageSkeleton.tsx` component already exists right next to them, just not imported/used. Done 2026-07-17 — *the page-level boundary (wraps the whole async page, fires before anything including the real breadcrumb/header has rendered) now uses the full `PageSkeleton` . Split a new `ResultsSkeleton` (sidebar + product grid only, no breadcrumb/header) out of it for the inner boundary around `InstantSearchSection` , so that one doesn't duplicate the real breadcrumb/header already on screen by the time it could suspend. Typecheck + lint clean; page verified live (200).*
-
- ✓ **Improve the PDP's Suspense fallback** — currently a generic pulsing box, not shaped like the final content. Done 2026-07-17 — *new `PdpSkeleton.tsx` mirrors the real two-column layout (breadcrumb, gallery + quick-specs stat bar on one side, title/price/CTA stack on the other). Close enough in shape for both real PDP layouts it can precede (`ProductVariantShell` for containers, `AccessoryDetail` for accessories) — exact per-layout fidelity isn't achievable from one shared fallback, but this is a meaningful improvement over the prior single pulsing div either way. Typecheck + lint clean; verified live against one of each product type (200).*
-

- ☐ **Decide the homepage's direction:** it's almost entirely static marketing content, including a `QuoteForm` whose submit button doesn't call any backend at all. Either wire it to a real lead-capture endpoint or explicitly accept it as decorative.

- ☒ **Decide between the two homepage reviews carousels** — `ReviewsSection.tsx` (static) and `ReviewsSectionLive.tsx` (real backend data) are intentionally rendered side by side right now for comparison; pick one before shipping. *Decided 2026-07-18 — retain both, for now. No code change; moved to §2 as a confirmed decision so future audits stop flagging this as pending.*

- ☐ **New 2026-07-17 — the PDP's "You May Also Need" section is fully fake, not just static.** `ProductDetail.tsx` renders a hardcoded 4-item `staticRelatedProducts` array with placeholder prices ("From \$2,000" , "Call for Price") and CTA buttons with zero `onClick` handlers — while the page **does** fetch real `related_products` from ES (`page.tsx`), which today is only consumed internally by `ProductInfoPanel` for variant-swapping, never rendered as an actual related-items display. Either wire the real data into this section or remove it — showing fabricated prices to real customers is worse than showing nothing.

- ☐ **New 2026-07-17 — `revalidateTag` / `updateTag` aren't wired to any real mutation.** `actions/cache.ts` 's `revalidateAll()` / `updateAll()` exist but are never called anywhere in the app (only referenced in AGENTS.md's own doc example); the only real callers of `revalidateTag` are the two webhook routes. `CACHE_TAGS.ORDERS` / `USERS` are declared in `config/cache.ts` but never attached via `cacheTag()` to anything, nor targeted by any revalidation call — meaning cart/checkout/profile mutations never bust their own cached data.

- ☒ **Dead `href="#"` consent-checkbox links in checkout.** `CheckoutClient.tsx` 's "Delivery Requirement," "terms and conditions," and "privacy policy" consent-checkbox links all went nowhere (`href="#"`). The real WordPress pages exist and already resolve (`/terms` , `/privacy`) — just needed pointing at them. *Done 2026-07-18 — "Delivery Requirement" now links to `/shipping-policy` (closest real match — its title is literally "Shipping Policy | Shipping Container Delivery"), "terms and conditions" to `/terms` , both "privacy policy" instances to `/privacy` . All 4 use `<Link target="_blank" rel="noopener noreferrer">` rather than a same-tab navigation, so clicking one mid-checkout doesn't lose in-progress form state. Verified live with a real cart item — all 4 links present with the correct hrefs and target="blank"._*

- ☒ **Raw `<img>` in `CheckoutClient.tsx`** (order-summary line-item thumbnail), carrying an `eslint-disable-next-line @next/next/no-img-element` that acknowledged but didn't fix the violation. *Done 2026-07-18 — swapped to `next/image` (fixed 56×56 thumbnail, matching its existing `h-14 w-14` container). Verified live: renders correctly with a real image URL. Typecheck + lint clean.*

Lower priority / needs a product decision first

- ☒ Product comparison — build the feature or remove the dead "Compare" button on the PDP. *Decided 2026-07-18 — no client requirement yet. Dead button removed (see `ProductInfoPanel.tsx` , commit `1356d1c`) rather than left non-functional.*

- ☒ Wishlist / save for later — build the feature (button + persistence) or remove the dead "Save to Wishlist" button. *Done 2026-07-18 — built client-side only, no backend exists for this. New `context/WishlistContext.tsx` (localStorage, reducer-based to stay hydration-safe — same pattern `CartContext` already uses) + `hooks/useWishlist.ts`. PDP's "Save to Wishlist" button now toggles real state (filled heart + "Saved to Wishlist" label when active). New `/wishlist` page (`ROUTES.WISHLIST`, robots-disallowed/noindexed like cart/checkout) lists saved items with "View Product" + "Remove" — deliberately no "Add to Cart" from this page, since a `WishlistItem` doesn't carry the full ES hit `CartItem.rawHit` needs for real backend cart sync; faking it would silently fail the same way the PDP's fake related-products section did. New `components/Layout/WishlistButton.tsx` (heart + count badge) sits next to `CartButton` in the Navbar for discovery regardless of login state — persistence is intentionally device-local only (no cross-device sync possible without a backend), called out directly in the page's own copy. Verified live end-to-end: toggle, localStorage persistence, nav badge count, wishlist page listing, remove- to-empty-state. Typecheck + lint clean (pre-existing unrelated errors confirmed via diff).*
- ☒ Social sharing — build or remove the dead "Share" button on the PDP. *Done 2026-07-18 — no backend needed at all, purely client-side. New `components/product/ShareButton.tsx`: uses the native Web Share API (`navigator.share`) on mobile/supporting browsers; falls back to a small dropdown (Facebook, X, WhatsApp, Email share intents + Copy Link) on desktop. `Lucide-react` doesn't ship Facebook/X brand icons, so those two rows use a generic external-link icon rather than a wrong/placeholder brand mark. Caught and fixed a real robustness gap while testing: `copyLink()` had no error handling — a headless-browser test confirmed `navigator.clipboard.writeText()` throwing (permission denied) would silently do nothing; wrapped in try/catch so it fails quietly instead of an unhandled rejection, with no fake "Copied!" confirmation shown on failure. Verified live: menu opens, all 4 share links carry the correct encoded product title/URL, copy-link shows real "Copied!" confirmation when clipboard access is granted, closes on outside click and Escape. Typecheck + lint clean.*
- ☐ Site-wide search UI — the ES backend already supports free-text query; just needs a search box added to the header/nav.
- ☒ Recently viewed products — net-new feature if wanted. *Decided 2026-07-18 — not required by the client. No action.*
- ☐ **Live chat — build or remove the marketing claim.** `TrustStrip.tsx` on the homepage explicitly advertises "Phone, Email & Chat" support, but no chat implementation exists anywhere. This is a false claim to customers today, worth fixing either way. *2026-07-18 — client hasn't decided on live chat itself yet; leaving both the feature and the marketing copy as-is until they do. Still an open false-claim risk in the meantime — revisit the copy specifically if the chat decision stalls long.*
- ☒ Multiple payment methods — the Braintree Drop-in is currently configured card-only; enabling PayPal/Venmo/Apple Pay may be cheap since the infra already exists. *Decided 2026-07-18 — Braintree card-only for now. No action.*
- ☐ Newsletter signup UI — backend utility functions already exist (`lib/newsletter.ts`); revisit building a signup form now that checkout has more real estate. *2026-07-18 — still pending a client decision, not resolved either way.*

- ☐ Wire the PDP's stock/availability badge to the real `variants[].qty` field instead of always showing a hardcoded "In Stock — Ready to Ship".

- ☐ Evaluate whether any data needs the `cacheLife('seconds')` tier — currently unused.

- ☒ Fix 2 default-import style violations (`braintree` in `payment.service.ts` , `cheerio` in `wp-proxy.service.ts`) — cosmetic/convention only, both server-only, no bundle impact. *Done 2026-07-18 — `payment.service.ts` now imports `{ BraintreeGateway, Environment, type Transaction }` ; `wp-proxy.service.ts` imports `{ load }` . `@types/braintree`` uses an `export = namespace`, so this needed care — verified it actually typechecks (it does, `esModuleInterop` is on) AND still works at runtime, not just compiles: `/api/braintree_token` returns the same "not configured" error as before (no import crash), and `/terms` (which exercises `cheerio.Load` through the WP proxy) still renders real content.*

- ☐ PLP ratings/review-count sync — already logged separately, see memory `backend-reminder-plp-ratings.md` (backend-side fix, not frontend).

- ☒ **New 2026-07-17 — Account deletion / data export has zero implementation** anywhere (no route, form, or backend call). A standard GDPR/CCPA-style expectation for a real storefront; needs a product decision on whether/how to build it, not just a code fix. *Decided 2026-07-18 — no requirement from the client. No action.*

- ☒ **New 2026-07-17 — Back-in-stock alerts and price-drop alerts are pure, total gaps** — no UI, hook, or backend call for either. Net-new features if wanted; not previously itemized explicitly (distinct from the general "notifications" area, which was previously only represented by the stock-badge/newsletter items above). *Decided 2026-07-18 — not required by the client; if built at all, it's expected to be a backend-driven subscriber notification (for users already subscribed), not a frontend feature. No action here.*

- ☐ **New 2026-07-17 — clarify who owns order confirmation emails.** No email-sending code (SendGrid/nodemailer/SMTP/etc.) exists anywhere in this Next.js app — if confirmation emails are sent today, it's entirely a Django-backend responsibility with zero visibility from here. Worth confirming with the backend team rather than silently assuming it's covered.

- ☒ **Dead "View All FAQs →" link on the PDP** (`ProductDetail.tsx` , `href="#"`). *Done 2026-07-18 — links to the real WordPress FAQ page (`/shipping-container-faqs` , confirmed live, real content) in a new tab. Verified live.*

- ☐ **PLP product cards never pass `priority`** to their `next/image` calls, including the first/likely-LCP card. Lower severity since the grid renders client-side via `react-instantsearch` rather than via SSR preload hints, but still worth setting on the first result. *Investigated 2026-07-18, deliberately not fixed — two real blockers, not just effort. (1) `ProductCard.tsx` (one of the two files originally named here) is confirmed dead code, never imported anywhere — the real live container cards render via `ProductHit` inside `InstantSearchSection.tsx`. (2) Both that component and the real, live `AccessoryCard.tsx` render through `react-instantsearch`'s `<Hits hitComponent={({ hit }) => ...}>`, whose render prop doesn't expose an item index — there's no clean way to know "this is the first card" without replacing `<Hits>` with a manual `useHits()` + `.map()` re-implementation, which risks regressing pagination/insights/empty-state behavior `<Hits>` currently provides for free, in exchange for a CWV nit the audit itself already scored as low-severity (client-rendered post-hydration, not part of the SSR preload path anyway). Left open rather than force a risky change to the PLP's core search UI.*

- ☒ **Cosmetic: stray `ref.md` scaffold files inside `app/`** (`app/(market)/cart/ref.md`, `app/(market)/my-account/ref.md`) — old component-spec notes from an earlier build pass, not real code, but violate the "`app/` holds only route files" convention in spirit. *Done 2026-07-18 — read both first: genuine historical build-spec/reference docs (one's a standalone design-mockup component, the other is written build instructions + starter code), not dead code — moved rather than deleted, matching this repo's own convention for this kind of doc (`MY_ACCOUNT_PROFILE.md`, `ABANDONED_CART_EXPLAINER.md`, etc.). Now `CART_PAGE_REF.md` and `MY_ACCOUNT_PAGE_REF.md` at the repo root.*

- ☒ **Cosmetic: `TrustedBySection.tsx` 's partner-logo alt text is generic** ("`Trusted partner logo 1`" – "`9`"), so authenticity of the social-proof claim can't be verified from the code alone — worth real, named alt text if these are genuine client logos. *Done 2026-07-18 — checked: the CDN filenames (e.g. `download-1-150x150-1.webp`) carry no real company names to attribute, so specific named alt text would have to be fabricated (worse than the placeholder). Used the actually-correct fix instead: `alt=""` on every logo — WCAG's recommended pattern for repeated decorative images inside a group that already has its own `aria-label` ("`Trusted organizations carousel`"), which already conveys the meaning. Documented why in a code comment so this doesn't read as an oversight later.*

- ☐ **New 2026-07-18 — the `ref.md` scaffold-file sweep on 2026-07-17 was incomplete.** Found by 2 independent research passes in this audit: 4 more stray reference files still live inside `app/`, same "`app/` holds only route files" violation as the 2 already moved out — `app/(market)/checkout/ref.md`, `app/(market)/sale-shipping-containers/ref.md`, `app/(market)/(home)/home-ref.html`, `app/(market)/(home)/home-ref-mobile.html`. Same nature (genuine historical build-spec/design-mockup docs, not dead code) and same fix applies — move to the repo root (e.g. `CHECKOUT_PAGE_REF.md`, `PLP_PAGE_REF.md`, `HOME_PAGE_REF.html`, `HOME_PAGE_REF_MOBILE.html`).

- ☐ **New 2026-07-18 — orphaned empty route scaffold:** `app/(market)/account/orders/` contains only a `.gitkeep`, present since the initial commit. It matches the *aspirational* folder structure documented in `AGENTS.md` (`app/(market)/account/orders/`) rather than the app's real, actually-used convention (`my-account/orders/`). Doesn't affect routing (no `page.tsx`), but could mislead a future contributor who reads `AGENTS.md` literally. Low severity — delete the empty folder.

- **New 2026-07-18 — Address book is not a true multi-address book.** `app/(market)/my-account/edit-address/_components/AddressForm.tsx` supports exactly one billing + one shipping address baked into the user profile (`ADDRESS_TYPES` is a fixed 2-entry array) — no add/remove/multiple-saved-addresses, no default-address selection. Fine as "edit your one address," not a real address book. Needs a product decision on whether true multi-address support is ever wanted before building it — would also need real backend support (the profile endpoint has no concept of multiple addresses today).

- **New 2026-07-18 — no shipment/carrier tracking number in Order History.** `my-account/orders` shows real order status (pending/paid/shipped/delivered/etc.) but no `tracking_number` /carrier field or link exists anywhere (confirmed via grep — zero matches). Distinct from the existing "order status updates" item, which is already real. Needs a product decision — likely also needs the backend to start returning a tracking number/carrier field, which it may not do today.

- **New 2026-07-18 — `Recaptcha.tsx` may be using the wrong `next/script` strategy.** It's rendered with `strategy="lazyOnload"` , but its only 2 real usages (`CheckoutClient.tsx` , `RegisterForm.tsx`) need the widget interactive *before* the user submits the form — not the deferred analytics/chat-widget case AGENTS.md's own rule 13 describes `lazyOnload` for. A fast-filling user could hit submit before the widget's finished loading. Worth trying `strategy="afterInteractive"` instead.